

# Deep Learning Python HW04

B1129046 楊家奎

## 1. 實作CNN模型

自訂的 CNN 架構包含兩個卷積區塊，接著是全連接層。每個卷積區塊使用 3x3 濾波器 (filter)、ReLU 激勵函數和 2x2 最大池化 (Max Pooling) 來減少空間維度並增加通道深度。

```
class CustomCNN(nn.Module):
    def __init__(self, dropout_p=0.2):
        super().__init__()
        # 區塊 1 : 1 個輸入通道 -> 32 個輸出通道
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1)
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)

        # 區塊 2 : 32 個輸入通道 -> 64 個輸出通道
        self.conv2 = nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3, padding=1)
        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)

        self.flatten = nn.Flatten()

        # 全連接分類器
        self.fc1 = nn.Linear(64 * 7 * 7, 128)
        self.dropout = nn.Dropout(dropout_p)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.pool1(torch.relu(self.conv1(x)))
        x = self.pool2(torch.relu(self.conv2(x)))
        x = self.flatten(x)
        x = torch.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)
        return x
```

為了監控訓練過程，我們將 PyTorch 的 SummaryWriter 整合到訓練迴圈中。這使我們能夠追蹤純量指標（訓練損失、測試損失和測試準確率）以及不同訓練週期 (epoch) 中模型權重的分佈。

```
# 初始化 writer
writer = SummaryWriter('runs/cnn_experiment_1')

# 在 epoch 迴圈內：
# 1. 紀錄純量 (損失與準確率曲線)
writer.add_scalar('Loss/Train', train_loss, epoch)
writer.add_scalar('Loss/Test', test_loss, epoch)
writer.add_scalar('Accuracy/Test', test_acc, epoch)

# 2. 紀錄直方圖 (權重分佈)
for name, param in model.named_parameters():
    if param.requires_grad:
        writer.add_histogram(f'Weights/{name}', param.data, epoch)
```

## 2. 結果與比較

透過使用 `param.numel()` 疊代模型參數，我們計算了總參數數量。與基準 DNN 相比，CNN 大幅減少了可訓練參數的數量。

| 指標            | 基準 DNN  | 自訂 CNN        |
|---------------|---------|---------------|
| 輸入處理          | 展平 (1D) | 空間 (2D 通道)    |
| 總參數數量         | 731,530 | 421,642       |
| 最終測試準確率       | 97.98%  | <b>98.88%</b> |
| 訓練週期 (Epochs) | 10      | 10            |

儘管參數數量比基準 DNN 少了約 57% (421,642 對比 731,530) ，CNN 仍達到了 [更高 / 相似] 的準確率。這證明了卷積神經網路在處理影像資料上的高效率。因為 CNN 使用權重共享 (weight sharing) (相同的濾波器在整張影像上滑動) ，它們需要少得多的參數就能學習複雜的特徵，從而在保持高效能的同時降低了過度擬合 (overfitting) 的風險。

### 3. Tensorboard 分析

Loss/Train (訓練損失) & Loss/Test (測試損失)：在 10 個訓練週期中，訓練損失曲線呈現穩定的下降趨勢，顯示模型透過梯度下降成功減少了誤差。測試損失緊緊跟隨訓練損失，沒有出現向上飆升的情況，這表明模型對未見過的資料具有良好的泛化能力，並沒有遭受嚴重的過度擬合。

Accuracy/Test (測試準確率)：準確率曲線在前幾個週期迅速攀升，隨後在接近最終值 98.88% 時趨於平緩。這種對數形狀是符合預期的，因為模型能快速學習明顯的特徵（如邊緣），並在後期的週期中微調更微小的細節。

